

一個有效解決 SDH 網路頻寬重組問題的演算法

張家榮¹ 張志清¹ 蔡英德²

¹靜宜大學資訊工程學系、²靜宜大學資訊傳播工程學系

摘要

傳輸網路的頻寬為電信網路經營者最重要的資產，頻寬的使用效率將直接影響網路營收及規劃策略。SDH 網路在管理上，因網路資源的應用缺乏完善的規劃或電路緊急開通與調度造成電路運作的管理混亂，傳輸線路因而產生頻寬碎片，導致電路使用效率低落。

頻寬重組可以解決頻寬碎片問題。

Acharya 等學者定義LDF問題來解決頻寬重組以移動頻寬的次數為頻寬重組的成本考量，他們也提出 Möbipack 演算法來解決此問題。然而，移動的總頻寬量在實務操作上也是重要的考慮因素。所以本研究提出 LDFB 問題，以移動的總頻寬量為新的成本考量。

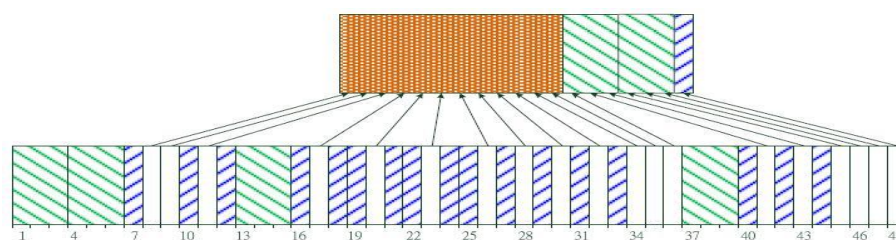
本研究提出 Best-Fit 方法解決 LDF 與 LDFB 問題，並經由模擬運算驗證此方法的效用性。實驗結果顯示 Best-Fit 在移動次數上與 Möbipack 接近，但在移動頻寬上顯著地超越 Möbipack 的結果。

一、引言

SDH(Synchronous Digital Hierarchy, SDH)是現代長途光纖通訊的標準，也是目前數位寬頻網路的骨幹。今日多數的電信經營者選擇以 SDH 架構作為建設網路主要技術是因為: SDH/SONET 建立一套標準化的光纖介

面，可以使不同設備商之設備互通，而不須配置任何轉換設備、具備高度的可擴充性、可靠性以及發展成熟的技術。對於以傳遞語音為主要業務的服務提供者而言，SDH 可以提供相當優異的傳輸時間延遲及網路故障保護機制，而對於逐漸成長的數據傳輸業務提供者而言 SDH 除了提供較佳的 QoS 機制及網路穩定性之外，相較於昂貴的 ATM (Asynchronous Transfer Mode)網路技術其高複雜性及乙太網路在 QoS 和故障保護機制的不足，選擇 SDH 技術做為建設基礎網路顯然要比 ATM 及乙太網路技術的投資成本及維護難度要低很多。

SDH 網路架構固然有其眾多的優點，但是 SDH 是一種以 TDM(Time Division Multiplexing) 為設計概念的傳輸模式，因此繼承了 TDM 的傳輸特性，始終無法擺脫網路資源使用率偏低的問題。當 SDH 網路進行電路的新增、刪除及異動等維護作業之後，常導致傳輸線路剩餘的時槽(Time Slot)處於非連續狀態，這個狀態將造成新的業務需求無法被配置，如圖一所示為 SDH 碼框結構等效為傳輸線路的時槽結構，其剩餘時槽總數雖足以提供一 STM-4(12ts)的業務需要，卻因為其時槽配置過於分散而無法完成。換言之，SDH 不允許將一個頻寬切割多等份後以分散的方式配置於傳輸線路。



圖一、傳輸線路的時槽結構

本研究主要是對 SDH 網路中，兩個 Node 間連接線路內部電路的配置做優化，其中 Node 是指 SDH 網路中的節點。並以 Swarup Acharya 等學者所提出的 Möbipack 演算法[6] 為基礎，但與其不同的是，我們以移動頻寬來當做優化成本考量的依據，因為在實際的情形下，想要求更大的頻寬必須要付出更高的成本，所以如果將小的頻寬與大的頻寬視為有不一樣的成本是比較合理的。本文其他部分組成如下，第二節先簡述 SDH 網路技術與傳輸頻寬優化問題，第三節說明 OMBW 演算法的內容與演算法的分析，第四節為實驗結果。

二、問題

在探討傳輸頻寬優化問題[1]之前，我們先說明 SDH 網路技術的幾個特性及在研究中須具備的幾點要求[2],[3],[5],[6]。

1. 電路格式

我們假設使用的電路是 SDH 網路中所定義的格式。可以為 VC-3, STM-1, STM-4, STM-16, STM-64 等，其中 VC-3 為本研究中使用最小的電路單位，佔一個時槽，而 STM- n 大小與 $3n$ 個 VC-3 相同，所以一個 STM-1 的容量是 3 個 VC-3，一個 STM-4 的容量是 12 個 VC-3。

2. 時槽映射特性

時槽的連續性:任何低階層速率之電路映射(Mapping)至SDH Frame時，該傳輸線路必須滿足對電路配置的時槽連續性 (contiguous concatenation)的要

求，意即頻寬無法被切割並配置於不連續的時槽區塊。為了提供日益劇增的數據傳輸SDH 發展另一種稱為Virtual concatenation 技術可提供較彈性的頻寬配置，主要作為Packet的應用服務。而本研究主要針對傳統的語音傳輸電路在配置效率的改善。

時槽映射位置:各類型的低階層速率電路只能被映射於高階層線路的特定時槽位址，低階電路在高階線路內所占用的時槽位於 s 到 $s+n-1$ ，且 $s = 1(\text{mod } n)$ 。其中 n 代表要映射的低階電路所佔的時槽大小，例如: VC-3 為 1，STM-1 為 3，STM-4 則為 12。例如，在 STM-16 的傳輸線路內 STM-4 只被允許配置起始於 1、13、25、37 時槽，並佔用 12 個時槽，STM-1 只能被配置起始於 1、4、7、10、13、...位址之時槽，由於 VC-3 被定義為最小的傳輸單位，只佔用 1 時槽，所以在線路的時槽配置上並無位址限制。

3. Bridge and Roll 功能

Bridge and Roll 為 SDH 所支援的一種無縫式路徑切換技術，主要用於 SDH 傳輸路徑異動時，為縮短影響服務時間而發展[4]。在無支援 Bridge and Roll 功能的 SDH 網路若產生電路修改需求時，其作業通常需要刪除欲異動的電路路徑資料後再建立一個全新的路徑，因此電路的異動過程將造成服務中斷，所造成的傷害除了資料封包或語音訊號的

丟失外，更嚴重的是造成客戶的抱怨及商譽的損失。本研究將假設所有網路節點都支援 Bridge and Roll 功能並且在網路無任何障礙的狀況下執行頻寬的異動。

LDF(Link Defragmentation)問題是在達成最佳化配置的情形下，並且要求移動頻寬個數最低。[6] LDFB(Link Defragmentation with Minimum Bandwidth Movement)，再達成最佳化配置的情形下，並且要求移動的頻寬總數最低。

定義：LDF(Link Defragmentation)問題

給定一個用來配置電路的 STM- n 線路(頻寬大小為 $link$)，一個電路的集合 $D = \{d_i \mid i \in [1, N]\}$ ，與一個是由電路映射到此電路所佔的時槽大小與其在線路中的起始位置的對應函數

$$F = \{(d_j \rightarrow \{g_j, s_j\}) \mid j \in [1, N], d_j \in D, g_j \in G, s_j \in [1, link]\}$$

。主要問題是找出一個新的對應函數

$$F' = \{(d_j \rightarrow \{g_j, s_j'\}) \mid j \in [1, N], d_j \in D, g_j \in G, s_j' \in [1, link]\}$$

，在滿足前述特性下並且同時滿足以下條件：

1. 最佳化配置
2. 移動的頻寬數量最低

最佳化配置是指運用最少的電路個數及電路速率種類，填滿線路內之所有時槽，使線路內無閒置時槽的狀況。

定義: LDFB(Link Defragmentation with Minimum Bandwidth Movement)問題

與 LDF 有相同給定的資訊，但是需要滿足的條件為：

1. 最佳化配置
2. 移動的頻寬總量最低

三、 Best-fit 演算法

本演算法的目的在於在花費最少優化成本的情形下，完成電路的最佳化配置。

為達成配置的優化，電路必須被遷移至

適當的位置，移出電路及移入電路的選擇方式將直接影響優化成本，衡量電路遷移的成本可由下列兩方面考量：

1. 電路的移動次數：由於被移動的電路皆處於服務狀態，所以移動次數愈頻繁，造成服務中斷的次數愈多，客訴及處理的頻率增加。
2. 移動頻寬的量：被移動的電路其頻寬越大則服務受影響的程度就越大，以被移動電路的時槽為單位，做為衡量遷移成本的準則，可以減少大頻寬電路被遷移的機會，對於服務中斷所造成的影響程度較低。

為了使頻寬的移動達到合理性及經濟性，我們以下列幾點作為移出頻寬及移入時槽選擇的必要條件：

1. 移入電路頻寬必須大於移出電路頻寬。
2. 設定電路權重做為選擇移動電路時之判斷依據，本研究預設電路權值依各種電路類型所占用之時槽而定，權值越小之電路則列為最優先選擇移出，STM-4 權值為 12，STM-1 權值為 3，VC-3 權值為 1。選擇移出之電路時，若兩個區段內被占用的電路頻寬總量相等時，則以所承載的電路權值較多之區段(電路個數較小者)為優先選擇。
3. 由於空閒的空間、與可填入的電路是給定的，而根據題目定義的最優化配置特性(Optimal Layout of Free Space, OLS)，移入的電路個數在一開始即可算得。

基於以上條件，以下將說明頻寬優化之方法與其步驟。演算法執行範例，參照附錄的演算法執行實例。

Input :

1. 頻寬的配置資料 D, F

2. 線路的頻寬 *link*

3. 所使用的電路大小 *G*

Output

1. 規劃好的移動清單 *ml* (*move list*)

2. 經過遷移後的空閒位子清單

fl (*freelist*)

演算法

PHASE 1. 初始化

PHASE 2. 產生移動清單

2.1 建立 Layout Tree

2.2 計算 OLS

2.3 建立 move list(*ml*)與 free
list(*fl*)

2.4 計算成本

PHASE 1: 初始化

令 *link* 為兩個 Node 間線路頻寬大小。

令 *G* 為所有電路的頻寬大小之集合，且
 $|G| = k$ 。

$$G = \{g_i \mid i \in [1, k], \forall m, n, g_m < g_n, \text{ if } m < n\}$$

令 r_i 為頻寬比值。

$$r_i = \begin{cases} link/g_i, & \text{if } i = k \\ g_{i+1}/g_i, & \text{if } i \neq k \end{cases}$$

令 *D* 為目前線路內所有的電路，且 $|D| = N$ 。

$$D = \{d_i \mid i \in [1, N]\}$$

令 *F* 為回傳目標電路所在位置與其使用頻寬
數之函數。

$$F: D \rightarrow \{G, [1, link]\}$$

$$F = \{(d_j \rightarrow \{g_j, s_j\}) \mid j \in [1, N], d_j \in D, g_j \in G, s_j \in [1, link]\}$$

PHASE 2: 產生移動清單

1. 建立 Layout Tree (*lt*)。其中每一節點表
示此節點所代表的相對應電路中有幾個電
路，與這些電路所佔有的頻寬，還有是否被保
留下來給移入電路使用。Node(count,

bandwidth, isMarked) 預設為 (0,0,false)。

1. 利用 *G* 建立空的 Layout Tree(*lt*)，其樹高
為 $h = |G| + 1$ 對每個 Level (*k-i*) 的節點
的 degree 為 r_i ，接著將每一節點的子節

點從左至右而且由 0 開始編號。

1.1 for (*d* in *D*) {

1.1.1 Let *pos* = (*F*(*d*) $\rightarrow$ *s*) - 1

1.1.2 Let *endCondition* = (*F*(*d*) $\rightarrow$ *g*)

1.1.3 Let *ptr* = *lt* 的根節點

1.2 for(*i* = *k* ; *i* >= 1 ; *i*--)

1.2.1 Let *tmp* = *pos* / *G*[*i*]

1.2.2 Let *ptr* = *tmp*-st child of *ptr*

1.2.3 if(*G*[*i*] == *endCondition*)

1.2.3.1 設定 *ptr* 節點的值

Node(1,*F*(*d*),*g*, false)

1.2.3.2 Break

1.3 利用 post order 去 run 這棵樹，只要
父節點沒有設定值，則此節點的值
等於其所有子節點的 Node 之合

2. 計算 OLS (Optimal Layout of Free
Space)。利用未使用空間計算最優化配
置。

2.1 Let *fs* (Free Space) = *link* - (根節點
的頻寬)

2.2 Let *ols* = new int[*k*]

2.3 for (*i* = *k* ; *i* > 0 ; *i*--)

2.3.1 *ols*[*k*] = *fs*/*G*[*k*]

2.3.2 *fs* -= *G*[*k*]**ols*[*k*]

3. 建立 move list 與 free list

3.1 Let *extra* = int[*k*] 預設為 0

3.2 For (令 *sections* 為 *lt* 中的 Node 的
集合，依照 *lt* 的 level *i* , *i* = [1,*k*])

3.2.1 將 Node 的頻寬與 *G*[*k-i*]一
樣的 mark 起來

3.2.2 排序 *sections* (依照頻寬遞增
排序，假如頻寬相等再依照個
數來遞增排序)(mark 起來的
要忽略)

3.2.3 *sections* 中選擇前
ols[*k-i*]+*extra*[*k-i*]個，將選擇
的 Node 加入 *fl* 中，並將其全
部 mark 起來。

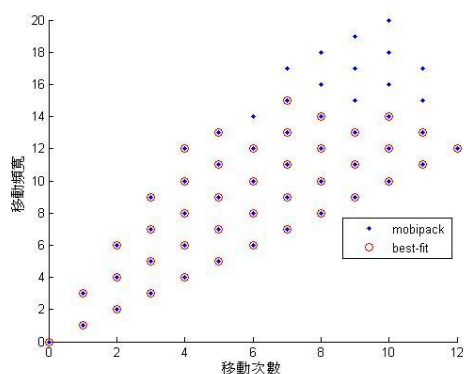
3.2.4 將其中所有電路填入 *ml*，並依照其個別的頻寬遞增 *extra* 相對應的位置。

4. 計算成本

四、結果分析

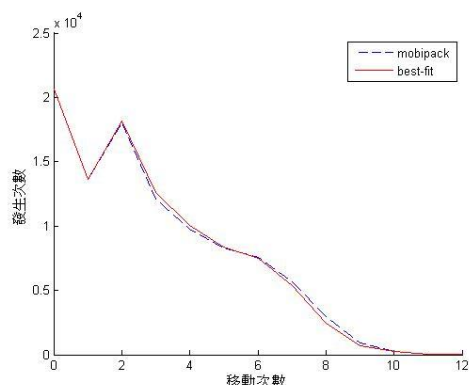
以下實驗為用亂數產生 100000 筆配置資料，並以 Möbipack 與 Best-Fit 兩個演算法執行優化後，分別記錄兩者之優化成本。其中線路大小為 STM-16，其中使用的電路為 {VC-3, STM-1, STM-4}。

圖一，可以看到 Möbipack 在某些情況中會略 BF 演算法多一些移動頻寬。



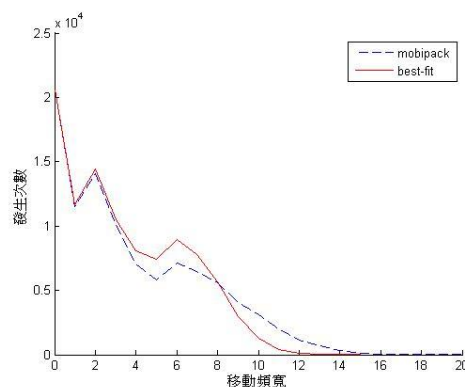
圖一、移動次數與移動頻寬關聯圖，線路為 STM-16。

圖二，在移動次數上面兩個演算法差距不大。



圖二、移動次數與發生次數關聯圖，線路為 STM-16。

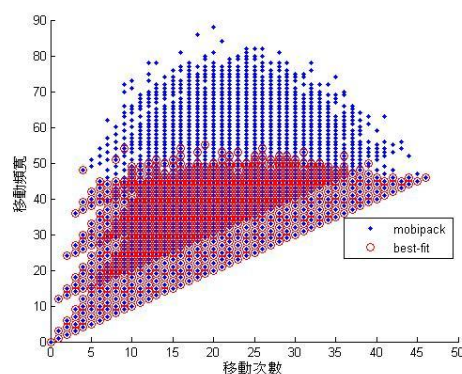
圖三，再移動頻寬量的比較上，BF 演算法已經開始看得出來有略勝於 Möbipack。



圖三、移動頻寬與發生次數關聯圖，線路為 STM-16。

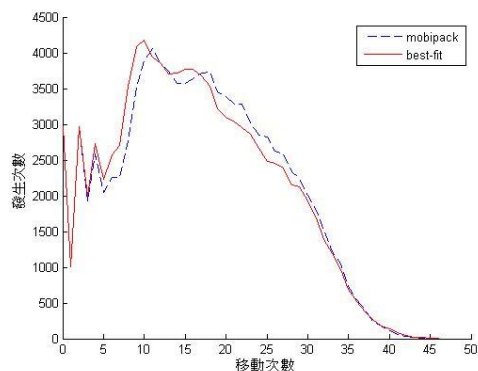
以下實驗為用亂數產生 100000 筆配置資料，其中通道大小為 STM-64，其中原本有的頻寬為 {VC-3, STM-1, STM-4, STM-16}。

在線路頻寬較大的情形中 BF 與 Möbipack 在移動次數上其實是相差不多的，但是可以更明顯的看出 Möbipack 的確會造成較多的移動頻寬成本。(圖四、圖五、圖六)



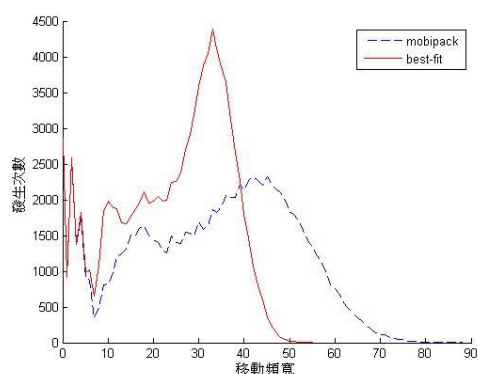
圖四、次數與頻寬關聯圖，線路為 STM-64。

圖五，在 möbipack 當作優化成本的移動次數上，OMBW 有與其十分接近的表現。



圖五、移動次數與發生次數關聯圖，線路為 STM-16。

圖六，OMBW 在移動頻寬上，比 mobipack 好，因為 mobipack 還會有移動超過 50 個頻寬的情形發生，而 OMBW 幾乎沒有這樣的情況。



圖六、移動頻寬與發生次數關聯圖，線路為 STM-16。

五、 結論

本研究藉由提出一個有效解決 SDH 網路頻寬重組最佳化問題的演算法，進一步的提升 SDH 網路的使用率。並透過優化使傳輸網路的資源潛力得到充分的發揮，不但節省投入成

本，對於網路的維護也更加具有彈性。

網路優化問題的範圍其實相當廣泛，加上新的網路技術不斷推陳出新，不同網路技術產生不同的問題，但是節省網路成本並提供穩定服務的目標是一致的，所以優化的問題永遠不會消失。本研究探討的 SDH 的技術特性與線路頻寬優化的問題，對於網路優化工作只是冰山一角，還有許多網路優化問題，諸如 NG-SDH 再數據傳輸的線路優化、ASON 對於自動頻寬配置優化等問題可以做進一步的研究與探討。

六、 參考文獻

- [1] D. E. Knuth, The Art of Computer Programming, Fundamental Algorithms, 2 ed. Addison-Wesley Pub Co, 1973, vol. 1.
- [2] GR-1230-CORE, "SONET Bidirectional Line Switched Ring Equipment Generic Criteria," Bellcore, November 1995.
- [3] W. Goralski, SONET, 2nd ed. McGraw-Hill Companies, 2000.
- [4] M.3100, "Definition of the management interface for a bridge and-roll cross-connect feature," Amendment 4, ITU-T, Aug 2001.
- [5] S. Acharya, B. Gupta, P. Risbood and A. Srivastava, "Hitless Link Defragmentation: Issues and Challenges," Bell Labs, Tech. Rep., 2003.
- [6] S. Acharya, B. Gupta, P. Risbood and A. Srivastava, "MöbiPack: Optimal Hitless SONET Defragmentation in Near-Optimal Cost", IEEE INFOCOM 2004, Vol. 3, pp. 1819-1829, 2004.

附錄

附錄一、演算法執行實例

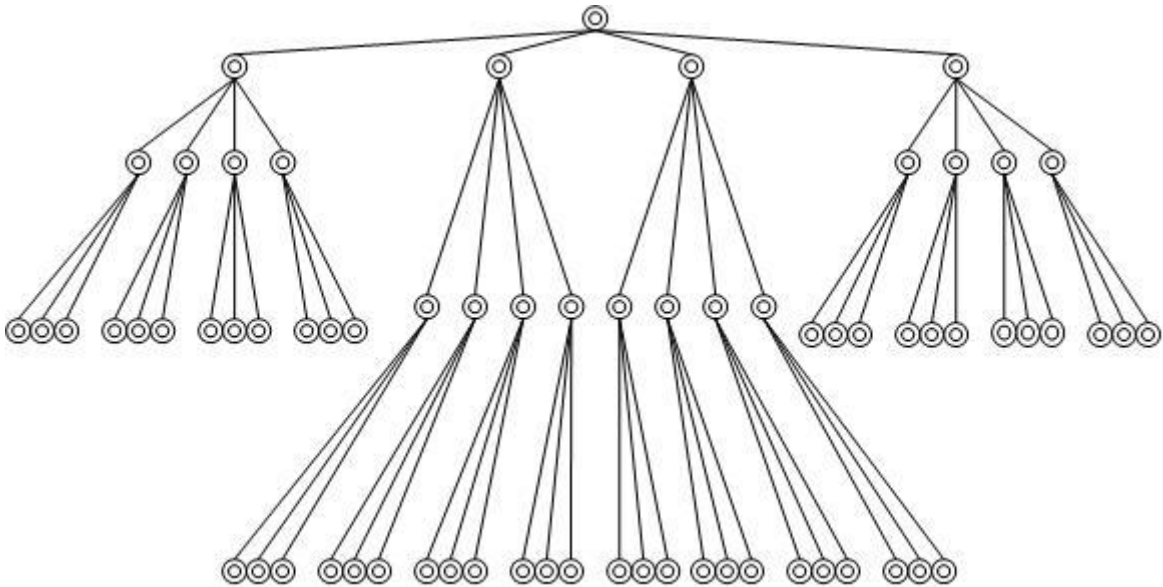
PHASH 2

1.1 建立 Layout Tree

Ex. $G = \{1,3,12\}$, link = 48

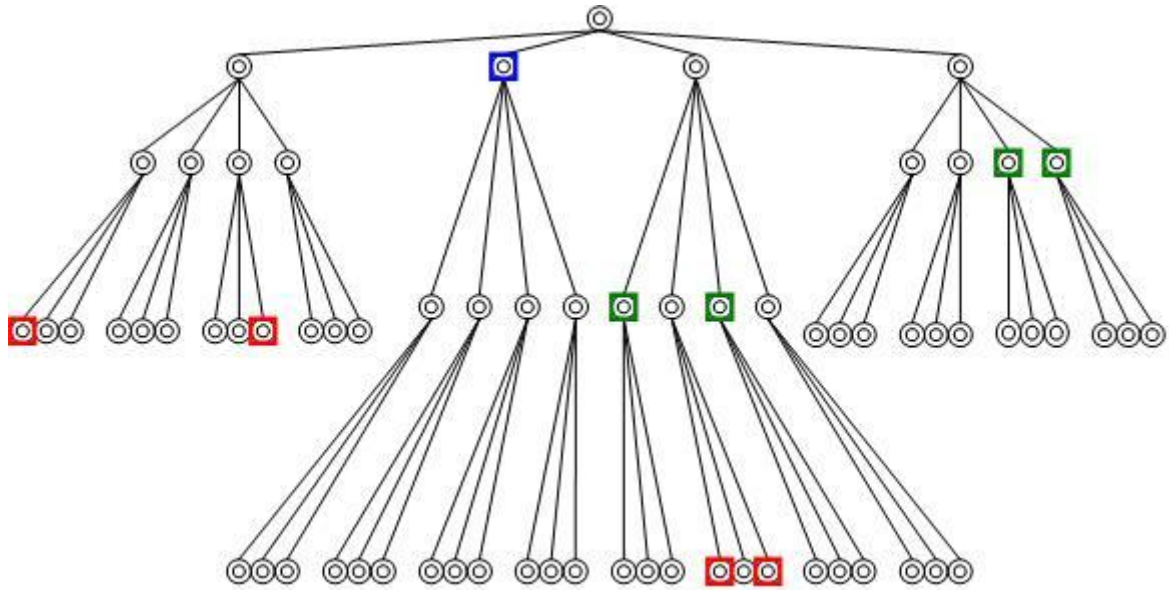
Height = 4

Level	degree
level 0	$r3 = 48/12 = 4$
level 1	$r2 = 12/3 = 4$
level 2	$r1 = 3/1 = 3$



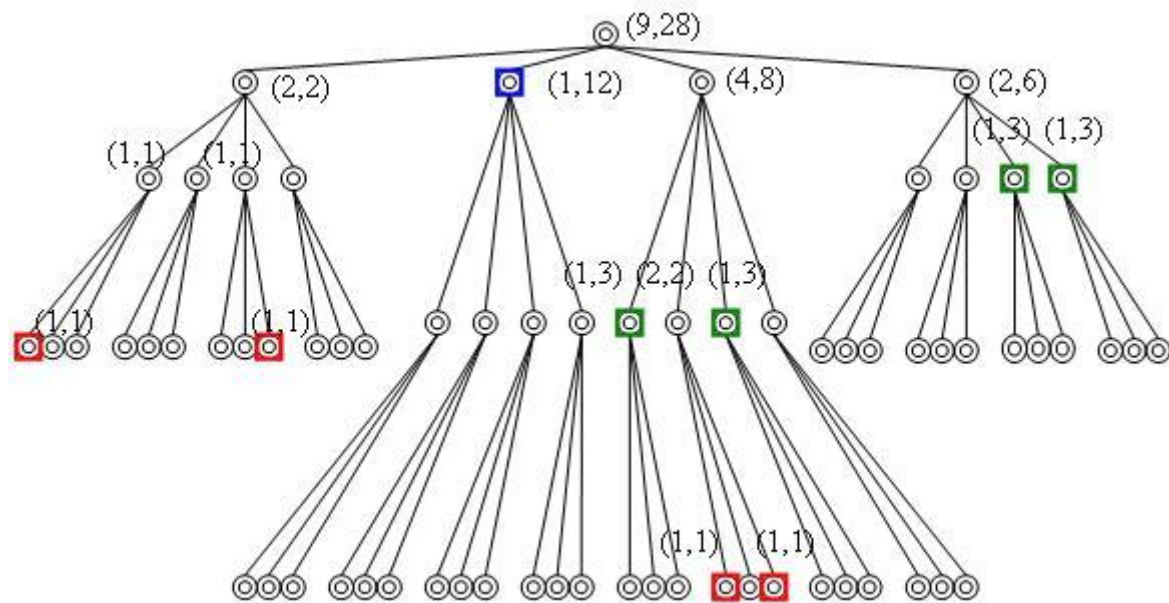
1.2 利用輸入的配置D與對應函數F，在 Layout Tree 上標記資訊

Ex . F: $\{\{d1,\{1,1\}\}, \{d2,\{1,9\}\}, \{d3,\{12,13\}\}, \{d4,\{3,25\}\}, \{d5,\{1,28\}\}, \{d6,\{1,30\}\}, \{d7,\{3,31\}\}, \{d8,\{3,43\}\}, \{d9,\{3,46\}\}\}$



1.3 計算 Layout Tree 上每一個 Node 的資訊

Ex.



2. 計算 OLS

Ex. 不使用的空間爲 20

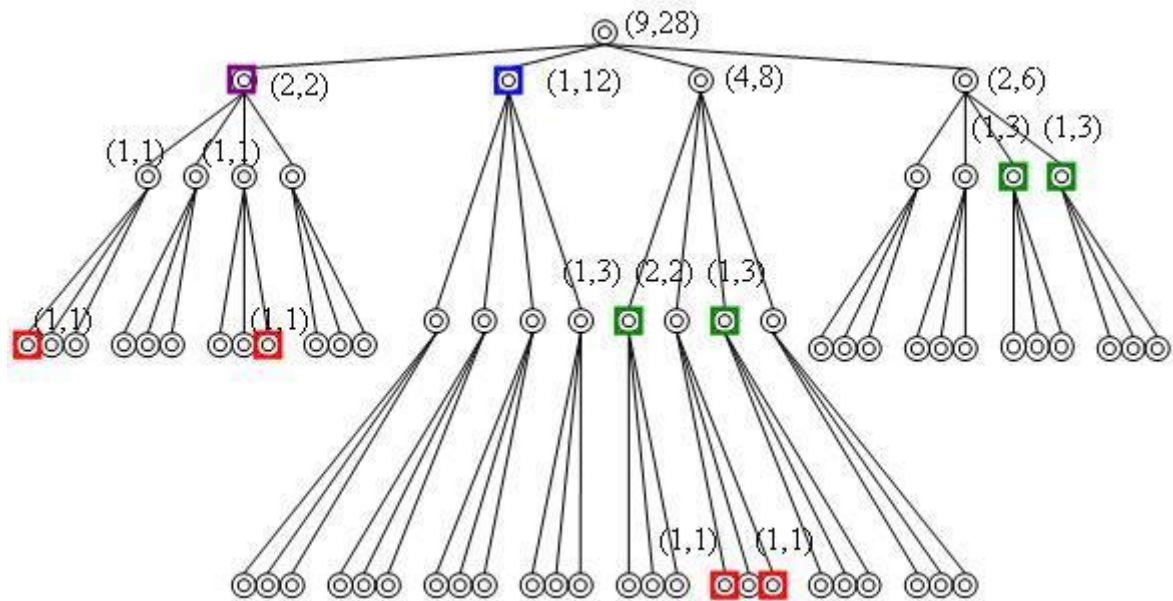
$$\text{OLS}[3] = 20 / 12 = 1 \dots\dots 8$$

$$\text{OLS}[2] = 8 / 3 = 2 \dots\dots 2$$

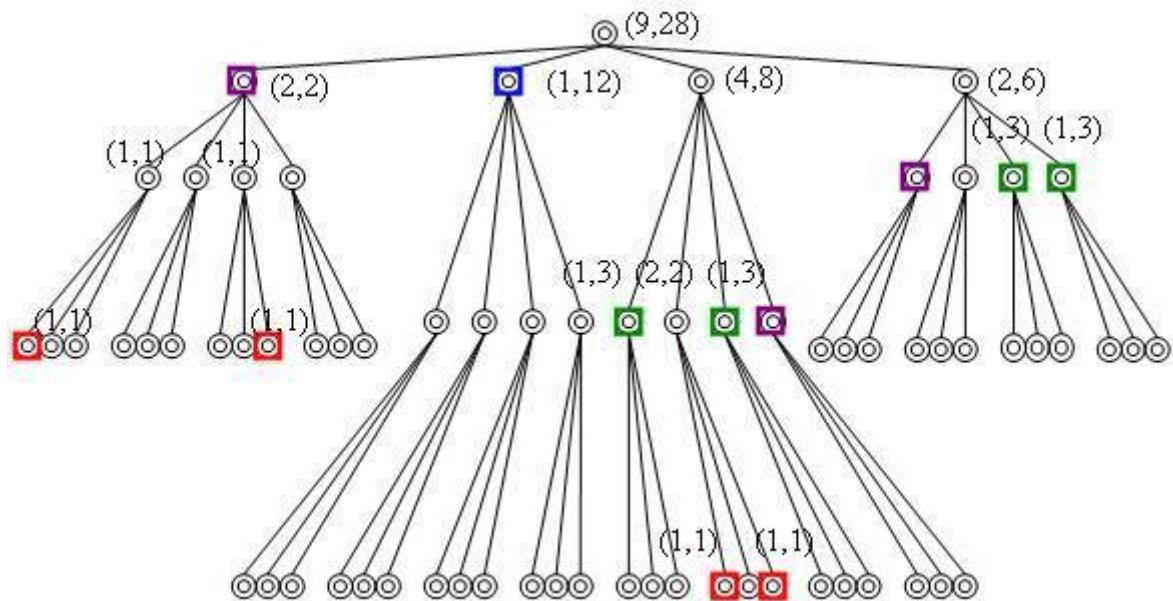
$$\text{OLS}[1] = 2 / 1 \dots\dots 1$$

$$\text{ols} = \{2,2,1\}$$

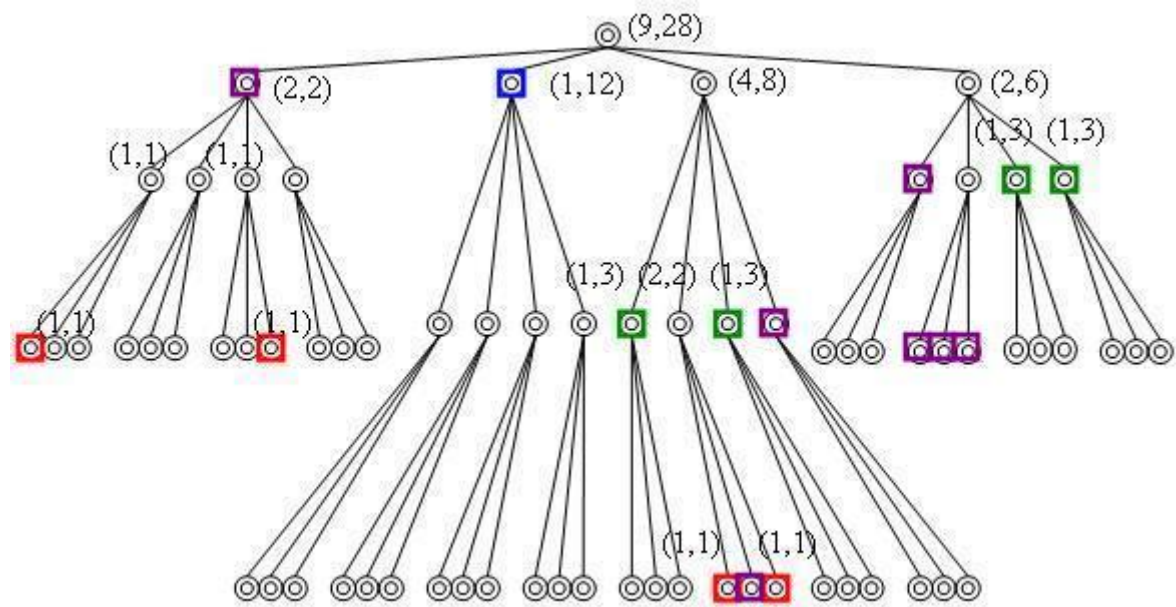
3 Ex. 建立 move list 與 free list



ols=[2,2,1] extra=[2,0,0]



ols=[2,2,1]extra=[2,0,0]



ols=[2,2,1] extra=[2,0,0]

PHASE 3

